

SGLang-Omni 每日 PR 学习日报

日期：2026-06-04 · 时间窗口：过去 24 小时合并 (merged) 的 PR · 仓库：[sgl-project/sglang-omni](https://github.com/sylphium-ai/project/sglang-omni)

15 个 PR 已合并

3 个深度讲解 (代码向)

主题：TTS 流式 / 音频编码 / ASR 基准

12 个文档 & 工程类

15

过去 24h 合并 PR 总数

3

多模态 / 多阶段核心代码 PR

~1800

核心代码新增行数 (合计)

Audio

本日绝对主线模态

当日概览

今天 SGLang-Omni 主仓在过去 24 小时内合并了 **15 个 PR**。绝大多数是文档刷新 (Higgs TTS / MOSS-TTS cookbook、logo、docker 发布说明等)，真正动到推理链路核心代码的有三个，且全部围绕音频模态 (**Audio**) ——这正是 SGLang-Omni 作为「Omni / 多模态多阶段流水线框架」当前的发力重点：

- **#655** — 给 Higgs TTS 加上原始 **PCM** 音频流式能力，并引入「首块更小」的 TTFC (首字节延迟) 调优旋钮。(信息量最大)
- **#686** — 用 **PyAV** 替换 **pydub**，给 TTS 输出加上 **opus / mp3 / aac** 编码，附带采样率自动重采样。
- **#647** — 修正 **Qwen3-ASR** CI 速度门槛 (concurrency 32 → 2)，并新增可复现的 **ASR** 并发扩展基准脚本。

本日报遵循「讲深 > 讲广」：上面三个吃透，其余 12 个在文末列成清单。每个深度卡片都按「概览 → 解决什么问题 → 具体改动 → 知识点 (小白向三段式) → 延伸思考」展开。卡片默认展开，点标题可折叠。

目录

1. [#655 · 为 Higgs TTS 增加原始 PCM 流式 + 首块调优旋钮](#)
2. [#686 · 基于 PyAV 的 mp3 / aac / opus 音频编码](#)
3. [#647 · 对齐 Qwen3-ASR CI 门槛 + 新增并发扩展基准](#)
4. [其余 12 个 PR \(文档 / 工程清单\)](#)
5. [附录 · 名词速查表](#)

Add PCM audio streaming controls for Higgs TTS

作者 @Hayden727 · 状态 **merged** · 合并于 2026-06-03 14:31 UTC · +953 / -69 · 15 文件 · [查看 PR](#) · [修复 #654](#)

1. 一句话概览

给 `/v1/audio/speech` 接口增加一条「原始 PCM 字节流」的输出通道（而不是只能走 SSE 包装），并新增一个 `initial_codec_chunk_frames` 旋钮，让 Higgs 的声码器（vocoder）把第一块音频切得很小，从而把「首声延迟」（TTFC）从 ~0.83s 降到 ~0.21s。

2. 它解决了什么问题

背景：Higgs TTS 是一个语音合成模型，用户发一段文字，服务端边生成边把音频流式推回来。在这个 PR 之前：

- 痛点 A：传输格式被 SSE 绑死。之前 `stream=true` 只能走 SSE（Server-Sent Events），也就是把音频块做 base64 编码塞进 `data: ...` 文本事件里。这对浏览器友好，但对「只想拿到裸音频字节直接喂给播放器/扬声器」的客户端来说，要多一层 base64 解码、体积也膨胀 ~33%。
- 痛点 B：响应头把采样率写死成 24kHz。之前裸 PCM 的元数据（采样率、声道、位深）是假设出来的，模型若换采样率就会播放错误（变速变调）。
- 痛点 C：首声太慢。声码器默认要攒够「一整个稳态块」（steady chunk）的码本帧才解码出第一段波形，导致用户按下播放后要等近 1 秒才听到声音——对实时语音助手是硬伤。

通俗类比：以前你点外卖（语音），只能让骑手把整袋饭装进一个标准保温箱（SSE+固定块）送来，且箱子上贴的「保温温度」标签是印死的。这个 PR 让你能选「不用保温箱、直接把饭递给我」（裸 PCM），标签按实物现写（动态响应头），还能让厨房「先炒一小口让你尝」（首块切小，TTFC 骤降）。

3. 具体做了什么改动

涉及 15 个文件，核心是 5 个：协议层、API 层、Higgs 声码器调度器、一个新的共享小工具，以及请求构造器。

3.1 协议层：新增 `stream_format` 与校验（`serve/protocol.py`）

在请求体模型 `CreateSpeechRequest` 上加了两个字段，并用 Pydantic 的 `model_validator` 做「组合参数互斥校验」：

```
class CreateSpeechRequest(BaseModel):
    ...
    stream: bool = False
```

```

stream_format: Literal["sse", "audio"] = "sse"    # ← 新：传输形态
initial_codec_chunk_frames: int | None = Field(default=None, ge=0)

@model_validator(mode="after")                    # 对象构造完后整体校验
def validate_stream_format(self):
    if self.stream_format == "audio":
        if not self.stream:
            raise ValueError('stream_format="audio" requires stream=')
        if self.response_format.lower() != "pcm":
            raise ValueError('stream_format="audio" requires response_format="pcm"')
    return self

```

逐行解读：`Literal["sse", "audio"]` 把字段限定为两个合法字符串，传别的值 Pydantic 直接报 422。`ge=0` 表示「greater-or-equal 0」，负数会被拒。

`@model_validator(mode="after")` 是在所有字段都填好后再跑的一段校验逻辑——这里强制：要走裸音频流，就必须 `stream=True` 且 `response_format=pcm`，否则报错。这叫「让非法状态不可表示」，把错误挡在入口而不是埋到下游。

3.2 API 层：从首块动态推导响应头 (`serve/openai_api.py`)

新增 `_speech_audio_response()`。它的精妙之处是：先消费流的第一块拿到真实采样率，再据此写响应头，然后把第一块和后续块一起吐出去。这样既不丢第一块，又能让响应头反映真实参数：

```

async def _speech_audio_response(client, gen_req, request_id, speed):
    """先读首块推导出 header，再开始真正的 PCM 流。"""
    chunk_stream = client.generate(gen_req, request_id=request_id)
    first_audio_bytes, stream_sample_rate = None, None

    # ① 先把流往前推，直到拿到第一段有效音频
    async for chunk in chunk_stream:
        if chunk.audio_data is None: continue
        first_audio_bytes, emitted_samples, stream_sample_rate = \
            _speech_pcm_chunk_bytes(chunk, emitted_samples=emitted_samples)
        if first_audio_bytes is not None: break

    async def _body():
        # ② 真正喂给 HTTP 响应体的生成器
        yield first_audio_bytes # 先把刚才“偷看”的首块还回去，不能丢
        async for chunk in chunk_stream:
            audio_bytes, emitted_samples, sample_rate = _speech_pcm_chunk_bytes(chunk, emitted_samples=emitted_samples)
            if sample_rate != stream_sample_rate: # ③ 中途变采样率→直接报错
                raise RuntimeError("Raw PCM stream sample rate changed .")
            yield audio_bytes

    return StreamingResponse(_body(), media_type="audio/pcm",

```

```
headers={"X-Sample-Rate": str(stream_sample_rate),
        "X-Channels": "1", "X-Bit-Depth": "16"})
```

关键技巧叫「**peek-then-replay**（先偷看再回放）」：HTTP 响应头必须在响应体第一个字节之前发出，可采样率只有看了第一块音频才知道。于是代码先 `break` 出来拿到首块和采样率、写好 header，再在 `_body()` 里先 `yield first_audio_bytes` 把偷看的那块补回去，保证不丢音频。③ 处「中途采样率变了就报错」是一种防御式契约：裸 PCM 没有容器头来描述格式变化，所以全程必须恒定采样率，否则播放端会错乱。

3.3 新增共享工具 `models/tts_streaming.py`

一个 37 行的小模块，定义了参数名常量和一个「解析+夹紧」函数，被 API 层和声码器层共享，避免魔法字符串散落各处：

```
INITIAL_CODEC_CHUNK_FRAMES_PARAM = "initial_codec_chunk_frames"

def resolve_initial_codec_chunk_frames(params, *, steady_chunk_frames: int)
    """返回请求级的首块码本帧数，并夹紧到不超过稳态块大小。"""
    if params is None: return 0
    value = params.get(INITIAL_CODEC_CHUNK_FRAMES_PARAM)
    if value is None: return 0
    frames = int(value)
    if frames < 0: raise ValueError(...)
    return min(frames, int(steady_chunk_frames)) # 夹紧：首块不能比稳态块还大
```

「夹紧 (**clamp**)」是关键守卫：如果用户要求的首块比正常稳态块还大，就没有「首块更小」的意义了，所以上限取稳态块大小。返回 0 表示「不启用首块特殊处理，走默认」。

3.4 声码器调度器：真正让首块变小 (`higgs_tts/vocoder_scheduler.py`)

这是性能收益的来源。声码器是「把模型生成的离散码本帧 (**codec frames**) 解码成 PCM 波形」的部件。原本它每次要攒够 `stream_stride` 行才解一次；现在对尚未发出过任何音频的请求，允许第一次只解 `initial_codec_chunk_frames` 帧：

```
steady_codec_frames = max(1, self._stream_stride - num_codebooks + 1)
use_initial_chunk = (
    state.initial_codec_chunk_frames > 0
    and state.initial_codec_chunk_frames < steady_codec_frames
    and not state.has_emitted # 只对“还没发过音频”的首块生效
)
first_decode_rows = max(num_codebooks,
                        state.initial_codec_chunk_frames + num_codebooks)
next_decode_rows = state.next_decode_rows or (
    first_decode_rows if use_initial_chunk and not is_final
```

```

else max(num_codebooks, self._stream_stride)) # 非首块→回到稳态步长
...
if use_initial_chunk and not is_final:
    emit_until_raw = min(raw_total, state.initial_codec_chunk_frames) #

```

逻辑对比——改动前：无论第几块，都要等满 `stream_stride` 帧才解码、才发声 → 首声慢。
 改动后：第一块只要攒够 `initial_codec_chunk_frames`（实验里设为 1）就立刻解码发声，
 之后立即回到稳态步长（见 `_next_decode_rows_after_emit`），保证只有「开头」被切
 小，后续吞吐不受影响。`num_codebooks` / 延迟模式（delay pattern）相关的
`+num_codebooks-1` 是 Higgs 多码本声学 token 的对齐补偿，后面知识点会讲。

3.5 实验数据（作者在 H200 上的 A/B 消融）

语种	模式	TTFC 均值	吞吐	首块字节
EN	SSE 默认	0.831s	13.02 qps	121,758
EN	裸 PCM 默认	0.823s	12.99 qps	115,759
EN	SSE + initial=1	0.219s	12.04 qps	1,920
EN	裸 PCM + initial=1	0.209s	12.23 qps	1,920

作者诚实的结论（很值得学）：「裸 PCM 传输本身」几乎不改善 TTFC（0.831→0.823，仅 -1%）；真正把 TTFC 砍掉 ~74% 的是 `initial_codec_chunk_frames=1`，它把首块从 12 万字字节缩到 1,920 字节。代价是：块数变多、整体吞吐略降、平均延迟略升——典型的「用吞吐换首响」权衡。作者特意在 PR 里更正了早期「PCM 提升性能」的误读，这是很好的工程严谨性。

4. 涉及的知识点（小白向三段式）

TTS / 声码器 (Vocoder)

它是什么：TTS (Text-To-Speech) 把文字变成语音。现代神经 TTS 通常两步：先由语言模型生成一串离散的「声学 token / 码本帧」，再由声码器把这些 token 解码成真正能听的 PCM 波形。

为什么需要它：直接让模型回归出几万个浮点采样点又慢又难训；先生成压缩的离散 token、再用专门的声码器还原，又快又稳。

本 PR 里：`HiggsStreamingVocoderScheduler` 就是这个「token→波形」的解码部件，PR 改的就是它「攒多少帧解一次」的节奏。

PCM 与「裸流 vs SSE」

它是什么：PCM（脉冲编码调制）是最原始的数字音频——一串 16 位整数采样点，没有任何文件头/容器。SSE (Server-Sent Events) 是一种基于 HTTP 的服务器单向推送文本协议，每条形如

```
data: ....\n\n
```

为什么需要区分：SSE 适合浏览器、能携带元数据，但要把二进制音频 base64 成文本（膨胀 33%、需解码）；裸 PCM 直接是字节，省 CPU、省带宽、播放端零解析。

本 PR 里：`stream_format="audio"` 选裸 PCM 通道，用 HTTP 响应头（`X-Sample-Rate` 等）替代 SSE 里的元数据字段。

TTFC (Time To First Chunk, 首块延迟)

它是什么：从客户端发出请求到收到第一段可播放音频的时间。

为什么需要它：对实时语音（语音助手、同传），用户对「开口快不快」极敏感；总时长可以稍长，但「按下后多久出声」决定体验。它和「总延迟 / 吞吐」是不同维度。

本 PR 里：`initial_codec_chunk_frames=1` 专门优化 TTFC，把首声从 0.83s 压到 0.21s——本质是「先挤一小口声音出来安抚用户，剩下的边播边算」。

码本帧 / 多码本 & 延迟模式 (delay pattern)

它是什么：Higgs 这类音频 codec 用多个码本 (**codebooks**) 并行表示同一帧音频（类似 RVQ，残差矢量量化）。为了让自回归模型更好生成，常用「延迟模式」——第 k 个码本比第 0 个码本晚 k 步出现，形成阶梯。

为什么需要它：多码本能用更少 token 表达高保真音频；延迟模式让模型在预测当前码本时能看到已生成的低层码本，质量更好。

本 PR 里：代码中反复出现的 `num_codebooks`、`+ num_codebooks - 1`、`reverse_delay_pattern` 就是在做解码前的对齐补偿——把阶梯状的码本「对齐回同一帧」才能正确还原波形。理解到「这是多码本对齐的偏移量」即可。

异步生成器 & StreamingResponse (背压式流式)

它是什么：Python 的 `async def` + `yield` 组成「异步生成器」，每次 `yield` 吐一块数据；FastAPI 的 `StreamingResponse` 把它接成 HTTP chunked 响应。

为什么需要它：音频是边生成边发，不能等全部算完再返回（否则首响极慢、内存爆）。生成器天然实现「算一块、发一块、再算下一块」的背压 (**backpressure**)。

本 PR 里：`_body()` 就是这样一个异步生成器，配合「先 break 偷看首块、再在 body 里 yield 回去」的技巧，兼顾了「响应头要早发」和「音频不能丢」。

5. 我能学到什么 / 延伸思考

- 可复用工程思想：(1)「让非法状态不可表示」——用 `model_validator` 在入口拒绝非法参数组合；(2)「peek-then-replay」——需要先看数据才能定头部时的通用套路；(3)「把旋钮做成请求级参数 + 服务端夹紧」——既灵活又安全；(4)「诚实的基准」——区分「PCM 传输」与「首块切小」两个变量，不把功劳乱归因，是真正的工程素养。
- 前置知识建议补：① 数字音频基础（采样率/位深/声道/PCM）；② RVQ / 多码本音频编解码（如 EnCodec、SoundStream）；③ 自回归 TTS 的 delay pattern（参考 MusicGen / Parler-

TTS) ; ③ HTTP chunked transfer 与 SSE 的差异 ; ④ FastAPI 的流式响应与 asyncio 生成器。

Add support for pyAV-based mp3, aac, opus audio encoding

作者 @haojin2 · 状态 **merged** · 合并于 2026-06-04 05:03 UTC · +289 / -20 · 6 文件 · [查看 PR](#) · 替代 #667

1. 一句话概览

把 TTS 输出的有损压缩编码 (mp3 / aac / opus) 从依赖 **pydub** (要外部 ffmpeg 命令) 改成用 **PyAV** (FFmpeg 的 Python 原生绑定) 实现, 并在采样率不被目标编码器支持时自动重采样到最接近的合法采样率。

2. 它解决了什么问题

- 痛点 A: **pydub** 依赖外部 **ffmpeg** 可执行文件。 **pydub** 本质是去 shell 里调用系统装的 **ffmpeg** 命令。容器里没装 ffmpeg、或路径不对, 编码就失败——部署脆弱、报错隐晦。
- 痛点 B: 采样率不合法直接崩。Opus 只接受 {8k,12k,16k,24k,48k}, AAC/MP3 也各有合法集合。模型若以 44.1kHz 出音, 直接喂给 Opus 编码器会报错。
- 痛点 C: **FLAC** 和有损格式的代码纠缠在一起。之前 FLAC (soundfile) 和 mp3/aac/opus (pydub) 混在一个分支里, 逻辑乱、回退路径不清晰。

类比: 以前你要烤面包 (编码音频), 得先确认厨房里装了一台特定型号的外置烤箱 (系统 ffmpeg), 没有就罢工。现在改用内置烤箱 (PyAV, 编译进 Python 包里), 自带温控; 面团尺寸不对 (采样率非法) 时还会自动帮你揉成合适大小 (重采样)。

3. 具体做了什么改动

核心文件 **sglang_omni/client/audio.py**。新增一张「编码配置表」+ 一个线性重采样函数 + 一个 PyAV 编码函数, 并重排了 **encode_audio** 的分支。

3.1 编码配置表: 声明式描述每种格式

```
PYAV_ENCODE_CONFIGS = {
    "opus": {"container": "ogg", "codecs": ("libopus", "opus"),
            "valid_rates": {8000, 12000, 16000, 24000, 48000}},
    "aac":  {"container": "adts", "codecs": ("aac",),
            "valid_rates": {8000, 16000, 24000, 44100, 48000, ...}},
    "mp3":  {"container": "mp3", "codecs": ("libmp3lame", "mp3"),
```

```

        "valid_rates": {8000, 16000, 24000, 44100, 48000, ...}},
    }

```

这是「配置即数据」的设计：每种格式用什么容器（ogg/adts/mp3）、优先用哪个编码器（libopus 失败就退回内置 opus）、支持哪些采样率，都写成数据，而不是写死成一堆 if-else。新增格式只要加一行表项。`codecs` 写成元组是「优先级回退列表」——先试更好的 `libopus/` `libmp3lame`，不可用再退到 FFmpeg 内置实现。

3.2 线性重采样：采样率不合法时自动救场

```

def _resample_linear(audio, orig_sr, target_sr):
    if orig_sr == target_sr: return audio
    duration = audio.shape[0] / float(orig_sr) # 原音频时长(秒)
    new_len = max(int(round(duration * target_sr)), 1) # 新采样点个数
    old_idx = np.arange(audio.shape[0])
    new_idx = np.linspace(0.0, audio.shape[0] - 1, num=new_len)
    return np.interp(new_idx, old_idx, audio) # 线性插值重采样

```

逐行：先按「时长 = 采样点数 / 采样率」算出音频持续多久；目标采样率下应有多少个点（`new_len`）；然后用 `np.interp` 在新位置上对旧样点做线性插值。这是最简单的重采样（不是最高音质，但零依赖、够用），目的只是把采样率「掰」到编码器能接受的值。

3.3 PyAV 编码核心

```

def _encode_with_pyav(audio, sample_rate, container_format, codecs, valid_rates):
    import av
    if sample_rate not in valid_rates: # ① 非法采样率→找最近
        nearest = min(valid_rates, key=lambda r: abs(r - sample_rate))
        audio = _resample_linear(audio, sample_rate, nearest); sample_rate = nearest
    buf = io.BytesIO(); container = av.open(buf, mode="w", format=container_format)
    stream = None
    for codec in codecs: # ② 编码器优先级回退
        try: stream = container.add_stream(codec, rate=sample_rate); break
        except av.CodecError: continue
    stream.layout = "mono"
    frame = av.AudioFrame.from_ndarray(audio.reshape(1, -1),
                                        format="f", layout="mono") # ③ 帧格式
    frame.sample_rate = sample_rate
    for packet in stream.encode(frame): container.mux(packet) # ④ 编码并打包
    for packet in stream.encode(None): container.mux(packet) # ⑤ flush
    container.close(); return buf.getvalue()

```

① 用 `min(..., key=abs差)` 找「最近的合法采样率」；③ FFmpeg 这些编码器要 `fltp` (`float planar`, 浮点平面) 格式的输入帧, 所以 `reshape` 成 `(1, N)`；④ `stream.encode(frame)` 把一帧 PCM 压成若干 packet, `container.mux` 把 packet 写进容器；⑤ 最后 `encode(None)` 是 FFmpeg 的惯例——传 `None` 触发编码器把内部缓冲里剩余的数据全部「冲刷 (flush)」出来, 否则结尾会丢音频。

3.4 分支重排 + 多级回退

新的 `encode_audio` 逻辑: mp3/aac/opus 优先 PyAV, 失败则回退 `pydub`, 再失败回退 `WAV`; `FLAC` 单独走 `soundfile` (失败回退 `WAV`)。这是一条「优雅降级 (`graceful degradation`)」链: 永远尽量给用户一个能播的东西, 而不是直接 500。

4. 涉及的知识点 (小白向三段式)

音频容器 vs 编解码器 (container vs codec)

它是什么: `codec` (如 `opus/aac/mp3`) 负责「怎么压缩声音」; `container` (如 `ogg/adts/mp4`) 是装这些压缩数据的「盒子」, 附带时间戳、元数据。

为什么需要区分: 同一个 `codec` 可装进不同 `container` (如 `AAC` 可在 `adts` 或 `mp4` 里)。播放器先拆盒子 (`demux`) 再解码。

本 PR 里: 配置表把二者解耦: `opus` → `ogg` 盒子、`aac` → `adts` 盒子、`mp3` → `mp3` 盒子, 编码时 `av.open(format=container) + add_stream(codec)` 分别指定。

PyAV vs pydub (绑定 vs 子进程)

它是什么: `pydub` 通过启动外部 `ffmpeg` 进程来干活; `PyAV` 是 FFmpeg 的 C 库 `Python` 绑定 (`Cython`), 把 `libav*` 直接编进 `wheel` 里。

为什么更好: 不依赖系统 `PATH` 里有没有 `ffmpeg`、不用为每次编码 `fork` 子进程 (更快)、错误能以 `Python` 异常精确捕获, 部署可复现性强。

本 PR 里: 正是为了去掉「外部 `ffmpeg` 可执行文件」这个隐性运行时依赖, 让容器化部署更稳。

采样率 & 重采样 (resampling)

它是什么: 采样率 = 每秒采多少个声音点 (`24000Hz` = 每秒 2.4 万点)。重采样 = 把一段音频从一个采样率换算到另一个。

为什么需要它: 不同编码器只支持特定采样率集合; 模型出音的采样率未必在其中, 硬编码会崩。

本 PR 里: 非法采样率时用 `np.interp` 线性插值「掰」到最近合法值, 保证编码不失败 (音质换稳健)。

优雅降级 / Fallback 链

它是什么：主路径失败时，自动退到次优但仍可用的路径，而不是直接报错。

为什么需要它：音频服务面向多样客户端，宁可给个 WAV 也别返回 500。

本 PR 里：PyAV → pydub → WAV 三级回退，每级都 `logger.warning` 记录，既稳又可观测。

5. 我能学到什么 / 延伸思考

- 可复用工程思想：(1)「配置即数据」把 if-else 沉淀成一张表，扩展性强；(2)「去隐性运行时依赖」用库绑定取代子进程调用，提升部署可复现性；(3)「多级 **fallback** + 显式日志」兼顾稳健与可观测；(4)「优先级回退列表」用元组顺序表达 codec 偏好。
- 前置知识建议补：① FFmpeg / libav 体系与 PyAV API (frame/stream/packet/mux)；② 数字信号处理里的重采样与抗混叠（本 PR 用的线性插值是简化版，进阶可看 soxr / librosa.resample）；③ 有损音频编码 (Opus/AAC/MP3) 的码率与采样率约束。

[CI] Align Qwen3-ASR stage-1 gate to concurrency=2 and add ASR concurrency-scaling benchmark

作者 @MelodyyyYin · 状态 **merged** · 合并于 2026-06-04 04:37 UTC · +561 / -217 · 4 文件 · [查看 PR](#) · 修复 #646

1. 一句话概览

修正 Qwen3-ASR 的 CI「速度门槛」：之前门槛是按 concurrency=32 测的数字，但 CI 实际只跑 concurrency=2，导致阈值松了 2-5 倍、抓不到回归；顺手把 CI 代码和一个新的「ASR 并发扩展基准」脚本共用同一条转写/打分路径，并把这条路径抽出来复用。

2. 它解决了什么问题

- 痛点 A：阈值与实测工况不匹配。CI 的 stage-1 门禁（`test-tts-ci.yaml` 里 `QWEN3_ASR_CI_CONCURRENCY=2`）实际以并发 2 运行，但 `test_qwen3_asr_ci.py` 里的速度基线（吞吐 10.295、延迟 0.193...）还是并发 32 的旧数字。并发 2 实测更快（吞吐 16.3、延迟 0.121），于是阈值松了 2-5 倍——性能回归了也亮绿灯，门禁形同虚设。
- 痛点 B：缺少可复现的并发扫描工具。issue #646 想要一张「不同并发下 WER / 吞吐 / 延迟 / RTF / 路由均衡」的矩阵来决策，但当时没有脚本能稳定产出。
- 痛点 C：CI 测试自己手写了一整套转写+打分逻辑（线程池、百分位、WER 汇总），和基准脚本重复，难维护、容易漂移。

类比：你给跑步机设了「时速低于 20km 报警」，但实际你只让它跑 5km 档位——这个 5km 档随便都超过 20，警报永远不响（门禁失效）。这个 PR 把报警线重设成贴合 5km 档的真实水平，并造了一台「能精确测各档位速度」的仪器（基准脚本），还让跑步机和仪器共用同一个测速探头（共享代码路径）。

3. 具体做了什么改动

3.1 重新标定阈值 + 默认并发改为 2（`tests/test_model/test_qwen3_asr_ci.py`）

把速度阈值改成并发 2 的真实测量值；把代码内默认并发从 32 改成 2，让本地跑、CI 跑、阈值校准（`tune-ci-thresholds`）三者对齐到同一工况。WER（识别准确率）阈值不变——因为 WER 与并发基本无关，并发只影响速度。

这里的洞察很关键：在 20 条样本的小工况上，「并发 2 比并发 32 还快」是正常的——样本太少时，高并发主要测的是「突发排队 / 尾延迟」，而非稳态吞吐；ASR 的 fan-out 只有在大工况上才划算。

3.2 新增基准脚本 `benchmarks/eval/benchmark_qwen3_asr_concurrency.py` (+494 行)

对一个正在运行的 **Qwen3-ASR router**，直接拿 SeedTTS 的参考音频去转写，按一组并发值扫描、每档重复 N 次，输出 WER / 墙钟时间 / 吞吐 / 延迟分位 / RTF / 每个 **worker** 的路由均衡。核心是一个异步发送函数：

```
def make_asr_send_fn(model_name, api_url, *, lang="en", max_new_tokens=1024):
    async def send_fn(session, sample) -> RequestResult:
        result = RequestResult(request_id=sample.sample_id)
        with open(sample.ref_audio, "rb") as f: audio_bytes = f.read()
        result.audio_duration_s = get_wav_duration(audio_bytes)
        form = aiohttp.FormData()
        form.add_field("model", model_name)
        form.add_field("language", lang)
        form.add_field("response_format", "json")
        form.add_field("file", audio_bytes, filename=..., content_type="audio/wav")
        start = time.perf_counter()
        async with session.post(api_url, data=form) as resp:
            payload = await resp.json()
            result.text = str(payload.get("text", "")); result.is_success = payload.get("success", False)
            result.latency_s = time.perf_counter() - start
            if result.is_success and result.audio_duration_s > 0:
                result.rtf = result.latency_s / result.audio_duration_s  # RTF
        return result
    return send_fn
```

这是个「闭包工厂」—— `make_asr_send_fn` 把 `model_name/api_url/lang` 固化进内层 `send_fn`，交给通用的 `BenchmarkRunner`（控制并发、warmup、超时）去并发驱动。注意代码注释里的坑：不要给 **Qwen3-ASR** 传 `temperature=0`，纯贪心会退化（服务端会偷偷把它抬到 0.01）。**RTF = 处理耗时 / 音频时长**，<1 表示比实时还快。

还有一段「路由均衡」探测，PR 里用它验证 router 是否把请求均匀分给各 worker：

```
def _worker_delta(before, after):
    # 读取 /workers 快照，算每个 worker 在本轮新增了多少 routed/success/failed
    for key in ("routed_requests", "successful_requests", "failed_requests"):
        deltas = {wid: after[wid] - before[wid] for wid in after}
        out[f"total_{key}"] = sum(deltas.values())
        if key == "routed_requests": out["per_worker_routed"] = deltas
    return out
```

3.3 共享路径复用：CI 测试直接 import 基准脚本

这是本 PR 最亮的结构改动。重构后，CI 测试删掉了自己那一坨手写逻辑（约删 -217 行），改成直接复用基准脚本的两个函数：

```
# test_qwen3_asr_ci.py 改动后
from benchmarks.eval.benchmark_qwen3_asr_concurrency import (
    build_asr_eval_results, run_asr_transcription)

outputs, wall_clock_s = asyncio.run(run_asr_transcription(
    seedtts_en_samples, port=..., concurrency=QWEN3_ASR_CONCURRENCY, war
results = build_asr_eval_results(seedtts_en_samples, outputs, wall_clock
corpus_wer = results["summary"]["corpus_wer"] # 然后只做 threshold 断言
```

「门禁 = 基准跑一遍 + 加阈值断言」——这是非常好的设计：CI 测的就是基准脚本本身那条路径，保证「线下基准」和「线上门禁」永远测同一件事，不会漂移。之前 CI 用 `concurrent.futures` 线程池手写并发，现在统一走基准的 `asyncio` + `aiohttp` 异步路径，连并发模型都对齐了。

4. 涉及的知识点（小白向三段式）

ASR（自动语音识别）& WER

它是什么：ASR 把语音转成文字（TTS 的逆过程）。WER（Word Error Rate，词错率）= (替换+删除+插入) / 参考词数，衡量识别准不准，越低越好。

为什么需要它：它是 ASR 质量的标准指标；CI 用 WER 门槛保证模型改动没把识别搞砸。

本 PR 里：用 `jiwer.process_words` 算 WER；并指出 WER 与并发无关，所以重标速度阈值时 WER 门槛保持不变。

并发 / Fan-out 与 RTF

它是什么：并发（concurrency / fan-out）= 同时在飞的请求数。RTF（Real-Time Factor，实时率）= 处理耗时 / 音频时长，<1 即快于实时。

为什么需要它：提高并发能在大工况上提升吞吐（GPU 吃满），但小工况上反而主要暴露排队延迟。RTF 让不同时长的音频可比。

本 PR 里：核心洞察——「小 20 样本门禁上并发 2 比 32 快是正常的，fan-out 只有大工况才划算」，所以门禁工况选 2，并用基准脚本扫描各并发量化验证。

Router & Worker / 路由均衡（DP=2）

它是什么：SGLang 推理服务常是「一个 router（路由器）+ 多个 worker（推理副本）」。Router 收请求、按负载分发给 worker；DP=2 表示 2 路数据并行副本。

为什么需要它：多副本能水平扩吞吐；但若路由不均，部分 worker 闲置、部分过载，整体被拖慢。

本 PR 里：`_worker_delta` 通过对比请求前后 `/workers` 快照，统计每个 worker 的 `routed/success/failed` 增量，验证路由是否均衡——这是诊断并发性能的重要维度。

CI 门禁 / 阈值校准 (regression gate)

它是什么：把关键指标 (WER、吞吐、延迟、RTF) 设阈值，PR 触发 CI 自动跑、超阈值就拦下——防止性能/质量回归悄悄进主干。

为什么需要它：推理框架改一行可能让吞吐掉 20%，人工很难每次发现；门禁是自动安全网。但阈值必须贴合实测工况，否则太松 (漏抓) 或太紧 (误报)。

本 PR 里：把阈值从并发 32 的数字重标到并发 2 的真实测量，并配 `tune-ci-thresholds` 技能，让基线可被自动、可复现地校准。

asyncio + aiohttp (异步并发 I/O)

它是什么：`asyncio` 是 Python 的协程并发框架，`aiohttp` 是其异步 HTTP 客户端。用单线程事件循环驱动成百上千个「等网络」的请求。

为什么用它 (替代线程池)：压测的瓶颈是「等服务端返回」(I/O 密集)，协程比线程更轻、上下文切换更省，能更精确地控制并发度。

本 PR 里：基准与 CI 统一用 `asyncio.run(run_asr_transcription(...))` + `aiohttp.FormData`，取代旧的 `concurrent.futures` 线程池，并发模型与基准对齐。

5. 我能学到什么 / 延伸思考

- 可复用工程思想：(1)「阈值要贴合实测工况」——基线数字必须来自 CI 真正运行的那个并发/工况，否则门禁是摆设；(2)「门禁 = 基准 + 断言」——让测试复用产品/基准代码路径，消除「测试和真实跑的不是同一套」的漂移；(3)「闭包工厂 + 通用 Runner」把「怎么发一条请求」和「怎么并发调度」解耦；(4)「先量化再决策」——用扫描数据支撑「并发选 2 还是 32」的结论。
- 前置知识建议补：① WER/CER 与 `jiwer`；② `asyncio` 协程、事件循环、`aiohttp`；③ 压测指标体系 (吞吐、p50/p95/p99、RTF、TTFC)；④ 推理服务的 router-worker 架构与负载均衡；⑤ 性能回归门禁与基线校准方法论。

其余 12 个 PR (文档 / 工程清单)

这些 PR 不改推理核心代码，多为文档刷新、CI/发布工程，按「讲深 > 讲广」原则不展开，列此备查。

#	标题	作者	类别	一句话
---	----	----	----	-----

690	[Docker] Release Formal Docker	@zhaochenyang20		发布正式 Docker 镜像，便于一键部署 Omni 服务。
688	[Docs] higgs-tts: correct language-tier counts (83/17)	@SilinMeng0510		修正 Higgs-TTS 文档里语言分层数量笔误。
685	[Docs] Update Higgs Audio Cookbook	@zhaochenyang20		更新 Higgs 音频 cookbook。
684	[Docs] Refresh Higgs TTS cookbook (align with model card)	@SilinMeng0510		让 cookbook 与模型卡一致。
683	[Docs] Quick Fix Higgs	@zhaochenyang20		Higgs 文档快速修正。
681	[Refresh] Higgs Performance	@zhaochenyang20		刷新 Higgs 性能数据表。
673	[Docs] Refresh Higgs TTS cookbook: voices, control-token, step-599 benchmarks	@SilinMeng0510		补充音色、控制 token 用法与最新基准。
670	Verify SGLang-Omni cookbook docs	@MelodyyyYin		校验 cookbook 文档可用性。
671	Docs/svg logo	@0xjeffro		更新 SVG logo 资源。
669	[Doc] Add MOSS-TTS in SGLang-Omni docs	@gaoyang07		把 MOSS-TTS 模型纳入文档体系。
668	Remove stale S2-Pro docs test	@xinlij		删除过时的 S2-Pro 文档测试。
652	[DOC] Add documentation in benchmark/README for newly-added flags	@haojin2		给 benchmark 新增的命令行 flag 补文档。

附录 · 名词速查表

术语	一句话解释
SGLang-Omni	面向「Omni / 多模态」模型的高性能多阶段流水线（ multi-stage pipeline ）推理框架；本日报中的音频链路就是其一条 pipeline。
多阶段流水线 (multi-stage)	把推理拆成若干 stage（如 文本 LM → 声学 token → 声码器），各 stage 可独立调度/并行，类似工厂流水线。
Tokenizer / Detokenizer Manager	SGLang 中负责把文本 ↔ token、以及把生成 token 还原成文本/音频的管理组件（本日 PR 未直接改动，但属同一架构家族）。
Scheduler（调度器）	决定哪些请求/哪些 token 在何时进入计算的核心组件；#655 改的 VocoderScheduler 是音频侧的流式调度器。
TTS / ASR	文字转语音 / 语音转文字，互为逆过程。本日主线全是音频模式。
Vocoder（声码器）	把离散声学 token 解码成 PCM 波形的部件。
PCM / SSE	原始字节音频 / 基于 HTTP 的文本事件推送协议（base64 包装音频）。
TTFC	首块延迟，实时语音体验的关键指标。
WER / RTF	词错率（识别质量） / 实时率（速度，<1 即快于实时）。
码本 / delay pattern	多码本并行表示音频 + 阶梯式延迟生成，提升音质、压缩 token 数。
PyAV	FFmpeg 的 Python 原生绑定，替代调用外部 ffmpeg 子进程。
Router / Worker / DP	路由器 + 多推理副本 + 数据并行，水平扩展吞吐的服务拓扑。
CI 门禁 / 阈值校准	用指标阈值自动拦截性能/质量回归；阈值必须贴合实测工况才有效。